Platform API Availability
FFmpeg API Implementation Status
Use with the `ffmpeg` command-line tool
VDPAU
VAAPI
DXVA2
VideoToolbox
Vulkan
CUDA (NVENC/NVDEC)
- NVENC
- NVDEC/CUVID
- cuda-nvcc and libnpp

libmfx (Intel Media SDK)
OpenCL
AMD UVD/VCE
- Decoding
- Encoding
- Transcode
- Transcode with Scaling

External resources

Many platforms offer access to dedicated hardware to perform a range of video-related tasks. Using such hardware allows some operations like decoding, encoding or filtering to be completed faster or using less of other resources (particularly CPU), but may give different or inferior results, or impose additional restrictions which are not present when using software only. On PC-like platforms, video hardware is typically integrated into a GPU (from AMD, Intel or NVIDIA), while on mobile SoC-type platforms it is generally an independent IP core (many different vendors).

Hardware decoders will generate equivalent output to software decoders, but may use less power and CPU to do so. Feature support varies – for more complex codecs with many different profiles, hardware decoders rarely implement all of them (for example, hardware decoders tend not to implement anything beyond YUV 4:2:0 at 8-bit depth for H.264). A common feature of many hardware decoders to be able to generate output in hardware surfaces suitable for use by other components (with discrete graphics cards, this means surfaces in the memory on the card rather than in system memory) – this is often useful for playback, as no further copying is required before rendering the output, and in some cases it can also be used with encoders supporting hardware surface input to avoid any copying at all in transcode cases.

Hardware encoders typically generate output of significantly lower quality than good software encoders like x264, but are generally faster and do not use much CPU resource. (That is, they require a higher bitrate to make output with the same perceptual quality, or they make output with a lower perceptual quality at the same bitrate.)

Systems with decode and/or encode capability may also offer access to other related filtering features. Things like scaling and deinterlacing are common, other postprocessing may be available depending on the system. Where hardware surfaces are usable, these filters will generally act on them rather than on normal frames in system memory.

There are a lot of different APIs of varying standardisation status available. FFmpeg offers access to many of these, with varying support.

Platform API Availability

	Linux			Windows			Android	Apple		Other
	AMD	Intel	NVIDIA	AMD	Intel	NVIDIA		macOS	iOS	Raspberry Pi
AMF	**Y**	N	N	**Y**	N	N	N	N	N	N
NVENC/NVDEC/CUVID	N	N	**Y**	N	N	**Y**	N	N	N	N
Direct3D 11	N	N	N	**Y**	**Y**	**Y**	N	N	N	N
Direct3D 9 (DXVA2)	N	N	N	**Y**	**Y**	**Y**	N	N	N	N
libmfx	N	**Y**	N	N	**Y**	N	N	N	N	N
MediaCodec	N	N	N	N	N	N	**Y**	N	N	N
Media Foundation	N	N	N	**Y**	**Y**	**Y**	N	N	N	N
MMAL	N	N	N	N	N	N	N	N	N	**Y**
OpenCL	**Y**	**Y**	**Y**	**Y**	**Y**	**Y**	*P*	**Y**	N	N
OpenMAX	*P*	N	N	N	N	N	*P*	N	N	**Y**

V4L2 M2M	N	N	N	N	N	N	*P*	N	N	N
VAAPI	*P*	**Y**	*P*	N	N	N	N	N	N	N
VDPAU	*P*	N	**Y**	N	N	N	N	N	N	N
VideoToolbox	N	N	N	N	N	N	N	**Y**	**Y**	N
Vulkan	**Y**	**Y**	**Y**	**Y**	**Y**	**Y**	N	N	N	N

Key:

- `Y` Fully usable.
- `P` Partial support (some devices / some features).
- `N` Not possible.

FFmpeg API Implementation Status

	Decoder			**Encoder**		**Other support**		
	Internal	Standalone	Hardware output	Standalone	Hardware input	Filtering	Hardware context	Usable from ffmpeg CLI
AMF	N	N	N	Y	Y	N	Y	Y
NVENC/NVDEC/CUVID	N	Y	Y	Y	Y	Y	Y	Y
Direct3D 11	Y	-	Y	-	-	F	Y	Y
Direct3D 9 / DXVA2	Y	-	Y	-	-	N	Y	Y
libmfx	-	Y	Y	Y	Y	Y	Y	Y
MediaCodec	-	Y	Y	Y	Y	-	N	N
Media Foundation	-	N	N	N	N	N	N	N
MMAL	-	Y	Y	N	N	-	N	N
OpenCL	-	-	-	-	-	Y	Y	Y
OpenMAX	-	N	N	Y	N	N	N	Y
RockChip MPP	-	Y	Y	N	N	-	Y	Y
V4L2 M2M	-	Y	N	Y	N	N	N	Y
VAAPI	Y	-	Y	Y	Y	Y	Y	Y
VDPAU	Y	-	Y	-	-	N	Y	Y
VideoToolbox	Y	N	Y	Y	Y	-	Y	Y
Vulkan	Y	-	Y	N	N	Y	Y	Y

Key:

- `-` Not applicable to this API.
- `Y` Working.
- `N` Possible but not implemented.
- `F` Not yet integrated, but work is being done in this area.

Use with the `ffmpeg` command-line tool

Internal hwaccel decoders are enabled via the `-hwaccel` option (now supported in ffplay). The software decoder starts normally, but if it detects a stream which is decodable in hardware then it will attempt to delegate all significant processing to that hardware. If the stream is not decodable in hardware (for example, it is an unsupported codec or profile) then it will still be decoded in software automatically. If the hardware requires a particular device to function (or needs to distinguish between multiple devices, say if several graphics cards are available) then one can be selected using `-hwaccel_device`.

External wrapper decoders are used by setting a specific decoder with the `-codec:v` (-c:v) option. Typically they are named `codec_api` (for example: `h264_cuvid`). These decoders require the codec to be known in advance, and do not support any fallback to software or other HW decoder if the stream is not supported.

Encoder wrappers are also selected by `-codec:v`. Encoders generally have lots of options – look at the documentation for the particular encoder for details.

Hardware filters can be used in a filter graph like any other filter. Note, however, that they may not support any formats in common with software filters – in such cases it may be necessary to make use of `hwupload` and `hwdownload` filter instances to move frame data between hardware surfaces and normal memory.

To get a list of the hwaccels available in the ffmpeg executable, use the command `ffmpeg -hwaccels`

VDPAU

Video Decode and Presentation API for Unix. Developed by NVIDIA for Unix/Linux systems. To enable this you typically need the `libvdpau` development package in your distribution, and a compatible graphics card.

Note that VDPAU cannot be used to decode frames in memory, the compressed frames are sent by libavcodec to the GPU device supported by VDPAU and then the decoded image can be accessed using the VDPAU API. This is not done automatically by FFmpeg, but must be done at the application level (check for example the `ffmpeg_vdpau.c` file used by `ffmpeg.c`). Also, note that with this API it is not possible to move the decoded frame back to RAM, for example in case you need to encode again the decoded frame (e.g. when doing transcoding on a server).

Several decoders are currently supported through VDPAU in libavcodec, in particular H.264, MPEG-1/2/4, and VC-1, AV1.

VAAPI

Video Acceleration API (VAAPI) is a non-proprietary and royalty-free open source software library ("libva") and API specification, initially developed by Intel but can be used in combination with other devices.

It can be used to access the Quick Sync hardware in Intel GPUs and the UVD/VCE hardware in AMD GPUs. See VAAPI.

DXVA2

Direct-X Video Acceleration API, developed by Microsoft (supports Windows and XBox360).

Several decoders are currently supported, in particular H.264, MPEG-2, VC-1 and WMV 3, AV1, HEVC.

DXVA2 hardware acceleration only works on Windows. In order to build FFmpeg with DXVA2 support, you need to install the dxva2api.h header. For MinGW this can be done by downloading the header maintained by VLC and installing it in the include path (for example in `/usr/include/`).

For MinGW64, `dxva2api.h` is provided by default. One way to install mingw-w64 is through a `pacman` repository, and can be installed using one of the two following commands, depending on the architecture:

```
pacman -S mingw-w64-i686-gcc
pacman -S mingw-w64-x86_64-gcc
```

To enable DXVA2, use the `--enable-dxva2` ffmpeg configure switch.

To test decoding, use the following command:

```
ffmpeg -hwaccel dxva2 -threads 1 -i INPUT -f null - -benchmark
```

VideoToolbox

VideoToolbox is the macOS framework for video decoding and encoding.

The following codecs are supported:

- Decoding: H.263, H.264, HEVC, MPEG-1, MPEG-2, MPEG-4 Part 2, ProRes
- Encoding: H.264, HEVC, ProRes

To use H.264/HEVC hardware encoding in macOS, just use the encoder `-c:v h264_videotoolbox` or `-c:v hevc_videotoolbox` for H.264 or HEVC respectively.

Check `ffmpeg -h encoder=...` to see encoder options.

VideoToolbox supports two types of rate control:

- Bitrate-based using `-b:v`
- Constant quality with `-q:v`. Note that the scale is 1-100, with 1 being the lowest and 100 the highest. Constant quality mode is only available for Apple Silicon and from ffmpeg 4.4 and higher.

Vulkan

Vulkan video decoding is a new specification for vendor-generic hardware accelerated video decoding. Currently, the following codecs are supported:

- Decoding: H.264, HEVC, AV1

The AV1 specification is currently an experimental specification developed in collaboration with the Mesa project. As such, it should not be expected to be implemented on any other drivers currently, but once an official specification is available, the decoder will be ported to use it.

To test decoding, use the following command:

```
ffmpeg -init_hw_device "vulkan=vk:0" -hwaccel vulkan -hwaccel_output_format vulkan -i I
```

Documentation on how to initialize the device, as well as filtering, is available on our documentation page.

CUDA (NVENC/NVDEC)

NVENC and NVDEC are NVIDIA's hardware-accelerated encoding and decoding APIs. They used to be called CUVID. They can be used for encoding and decoding on Windows and Linux. FFmpeg refers to NVENC/NVDEC interconnect as CUDA.

NVENC

NVENC can be used for H.264 and HEVC encoding. FFmpeg supports NVENC through the `h264_nvenc` and `hevc_nvenc` encoders. In order to enable it in FFmpeg you need:

- A supported GPU

- Supported drivers for your operating system
- The NVIDIA Codec SDK or compiling FFmpeg with --enable-cuda-llvm
- `ffmpeg` configured with `--enable-ffnvcodec` (default if the nv-codec-headers are detected while configuring)

> **Note:** FFmpeg uses its own slightly modified runtime-loader for NVIDIA's CUDA/NVENC/NVDEC-related libraries. If you get an error from `configure` complaining about missing `ffnvcodec`, this project is what you need. It has a working Makefile with an install target: `make install PREFIX=/usr`. FFmpeg will look for its pkg-config file, called `ffnvcodec.pc`. Make sure it is in your `PKG_CONFIG_PATH`.
>
> This means that running the following before compiling ffmpeg should suffice:
>
> ```
> git clone https://git.videolan.org/git/ffmpeg/nv-codec-headers.git
> cd nv-codec-headers
> make
> sudo make install
> ```

After compilation, you can use NVENC.

Usage example:

```
ffmpeg -i input -c:v h264_nvenc -profile high
```

t a g s

amf cuda direct3d dxva2 hwaccel mediacodec mmal nvenc opencl quick sync va-api vce vdpau videotoolbox vulkan

You can see available presets (including lossless for both hevc and h264), other options, and encoder info with `ffmpeg -h encoder=h264_nvenc` or `ffmpeg -h encoder=hevc_nvenc`.

> **Note:** If you get the `No NVENC capable devices found` error make sure you're encoding to a supported pixel format. See encoder info as shown above.

NVENC can accept d3d11 frames context directly.

```
ffmpeg -y -hwaccel_output_format d3d11 -hwacc
hevc_nvenc out.mp4
```

t a g s

amf cuda direct3d dxva2 hwaccel mediacodec mmal nvenc opencl quick sync va-api vce vdpau videotoolbox vulkan

NVDEC/CUVID

NVDEC offers decoders for H.264, HEVC, MJPEG, MPEG-1/2/4, VP8/VP9, VC-1, AV1. Codec support varies by hardware (see the GPU compatibility table).

Note that FFmpeg offers both NVDEC and CUVID `hwaccel`s. They differ in how frames are decoded and forwarded in memory.

The full set of codecs being available only on Pascal hardware, which adds VP9 and 10 bit support. The note about missing `ffnvcodec` from NVENC applies for NVDEC as well.

Sample decode using CUDA:

```
ffmpeg -hwaccel cuda -i input output
```

Sample decode using CUVID:

```
ffmpeg -c:v h264_cuvid -i input output
```

FFplay supports older option -vcodec hevc_cuvid, but not -c:v hevc_cuvid (though support for -hwaccel was recently added).

```
ffplay -vcodec hevc_cuvid file.mp4
```

Full hardware transcode with NVDEC and NVENC:

```
ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i input -c:v h264_nvenc -preset slow
```

AV1 NVDEC HW decoding requires using -c:v av1:

```
ffmpeg -hwaccel nvdec -c:v av1 -i input_av1.mp4 output.ts
```

An example using scale_cuda and encoding in hardware, scale_cuda is available if compiled with ffnvcodec and `--enable-cuda-llvm` (default is on, requires nvidia llvm to be present at runtime):

```
ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i file.mkv -noautoscale -filter_compl
```

another example

```
ffmpeg -hwaccel_device 0 -hwaccel cuda -hwaccel_output_format cuda -i input -vf scale_c
```

The `-hwaccel_device` option can be used to specify the GPU to be used by the hwaccel in ffmpeg.

cuda-nvcc and libnpp

```
  --enable-libnpp          enable Nvidia Performance Primitives-based code [no]
  --enable-cuda-nvcc       enable Nvidia CUDA compiler [no]
```

Both of these are basically "older" cuvid options that require the nvidia SDK to be present when compiled and run. libnpp provides scale_npp (and a few other _npp filters). They might have different options/flexibility than their XX_cuda equivalent. Might be similar performance. These have more funky licensing (nonfree). cuda-nvcc has basically been replaced with ffnvcodec + cuda-llvm, scale_npp with scale_cuda.

Example:

```
ffmpeg -hwaccel cuda -i input -vf scale_npp=-1:720 -c:v h264_nvenc -preset slow output.
```

libmfx (Intel Media SDK)

libmfx is a proprietary library from Intel for use of Quick Sync hardware on both Linux and Windows. On Windows it is the primary way to use for decoding, video processing and encoding beyond those accessible via DXVA2/D3D11VA. On Linux it provides a different and mostly wider range of features compared to VAAPI, specifically for encoding and often better performance.

See QuickSync.

OpenCL

OpenCL can be used for a number of filters. To build, OpenCL 1.2 or later headers are required, along with an ICD or ICD loader to link to - it is recommended (but not required) to link with the ICD loader, so that the implementation can be chosen at run-time rather than build-time. At run-time, an OpenCL 1.2 driver is required - most GPU manufacturers will provide one as part of their standard drivers. CPU implementations are also usable, but may be slower than using native filters in ffmpeg directly.

OpenCL can interoperate with other GPU APIs to avoid redundant copies between GPU and CPU memory. The supported methods are:

- DXVA2: NV12 surfaces only, all platforms.
- D3D11: NV12 textures on Intel only.
- VAAPI: all surface types.
- ARM Mali: all surface types, via DRM object sharing.
- libmfx: NV12 surfaces only, via VAAPI or DXVA2.

AMD UVD/VCE

AMD UVD is usable for decode via VDPAU and VAAPI in Mesa on Linux. VCE also has some initial support for encode via VAAPI, but should be considered experimental.

On Windows, UVD is accessible via standard DXVA2/D3D11VA APIs, while VCE is supported via AMF. The Advanced Media Framework (AMF) SDK provides developers with easy access to AMD GPUs for multimedia processing.

AMF is effectively supported by FFmpeg to significantly speed up video encoding, decoding, and transcoding via AMD GPUs.

Decoding

AMD supports hardware decoding via DirectX in FFmpeg. Currently supports DX9 and DX11 in FFmpeg.

Hardware decoding via DX9

```
ffmpeg -hwaccel dxva2 -i input.mkv output.yuv
```

Note: Currently AMD hardware doesn't support **AV1 elementary stream decoding via DX9**. So, this command line is not applicable for AV1 bitstream as input.

Hardware decoding via DX11

```
ffmpeg -hwaccel d3d11va -i input.mkv output.yuv
```

In the above command line, "input.mkv" is only an example. The AMD hardware accelerated decoder supports most widely used containers and video elementary stream types. The following table lists detailed information about the widely used containers and video elementary streams which the AMD hardware accelerated decoder supports.

Table: Containers and video elementary streams supported by the AMD hardware accelerated decoder

Format	Filename Extension	H.264/AVC	H.265/HEVC	AV1
Matroska	.mkv	Y	Y	Y
MPEG-4 Part 14 (MP4)	.mp4	Y	Y	Y

Audio Video Interleave (AVI)	.avi	Y	N	Y
Material Exchange Format (MXF)	.mxf	Y	n/a	n/a
MPEG transport stream (TS)	.ts	Y	Y	N
3GPP (3GP)	.3gp	Y	n/a	n/a
Flash Video (FLV)	.flv	Y	n/a	n/a
WebM	.webm	n/a	n/a	Y
Advanced Systems Format (ASF)	.asf .wmv	Y	Y	Y
QuickTime File Format (QTFF)	.mov	Y	Y	n/a

Key:

- 'Y': Hardware accelerated decoder supports this input
- 'N': Hardware accelerated decoder doesn't support this input
- 'n/a': This input is not applicable in specification

Encoding

Currently AMF encoder supports H.264/AVC, H.265/HEVC, AV1 encoder. FFmpeg uses `_amf` as the postfix for the AMF encoder names. The command lines shown below may use `h264_amf`, and should be replaced by `hevc_amf` for H.265/HEVC encoder and `av1_amf` for AV1 encoder.

```
ffmpeg -s 1920x1080 -pix_fmt yuv420p -i input.yuv -c:v h264_amf output.mp4

ffmpeg -s 1920x1080 -pix_fmt yuv420p -i input.yuv -c:v hevc_amf output.mp4

ffmpeg -s 1920x1080 -pix_fmt yuv420p -i input.yuv -c:v av1_amf output.mp4
```

Transcode

There are two possible methods for transcoding: hardware decoding and hardware encoding, or software decoding and hardware encoding.

Hardware Decode and Hardware Encode

Use DX9 hardware decoder

```
ffmpeg -hwaccel dxva2 -hwaccel_output_format dxva2_vld -i input.mkv -c:v av1_amf output
```

Use DX11 hardware decoder

```
ffmpeg -hwaccel d3d11va -hwaccel_output_format d3d11 -i input.mkv -c:v hevc_amf output.
```

The parameter `hwaccel_output_format` will specify the raw data (YUV) format after decoding.

To avoid raw data copy between GPU memory and system memory, use `-hwaccel_output_format dxva2_vld` when using DX9 and use `-hwaccel_output_format d3d11` when using DX11. This will improve transcoding speed greatly. **This is the best setting we recommend for transcoding.**

Software Decode and Hardware Encode

Use the CPU to decode the input bitstream, and the GPU to encode the output stream.

```
ffmpeg -i input.mkv -c:v av1_amf output.mp4
```

The default software decoder corresponding to the elementary video stream will be used as the decoder.

Transcode with Scaling

Scaling is a very common operation in transcoding. It is done through video filter in FFmpeg.

Hardware decode and hardware encode with scaling

```
ffmpeg -hwaccel d3d11va -i input.mkv  -vf scale=1280x720 -c:v h264_amf output.mp4
```

If filter parameters are used in transcoding, users can't set `hwaccel_output_format` parameters. In fact, the filter processing is finished in the CPU in the above example.

Software decode and hardware encode with scaling

In the following command line, both decoding and scaling are done via the CPU, and encoding is done via the GPU.

```
ffmpeg -i input.mkv -vf scale=1280x720 -c:v h264_amf output.mp4
```

External resources

- multimedia.cx: How To Play Hardware Accelerated Video on A Mac
- libav-user: Hardware accelerated decoding
- Stack Overflow: How to use hardware acceleration with ffmpeg

Last modified on Oct 28, 2024, 6:39:23 PM

t a g s

amf cuda direct3d dxva2 hwaccel
mediacodec mmal nvenc opencl quick sync
va-api vce vdpau videotoolbox vulkan